



Advanced Python programming using AI tools

Lecture 6: Modules, Exceptions & Files

Lecturer: Dr. Havana Rika

Today

- ❖ Modules
- ❖ Math
- ❖ Random
- ❖ Conditional Execution
- ❖ Exceptions
- ❖ Files

Modules

- **Module** - a **file** containing a **set of functions** and **variables** you want to include in your application.
- To **create** a **module** just save the **code** you want in a **file** with the file extension **.py**

mymodule.py

```
def greeting(name):  
    print("Hello, " + name)  
  
x=7
```

```
import mymodule  
  
mymodule.greeting("Jonathan")  
  
print(mymodule.x)  
mymodule.x = 5  
print(mymodule.x)
```

Import a Module

- We can **import** a module using the **import statement** and access the definitions inside it using the **dot** operator.

example.py

```
y=8

def func1(x):
    return 2*x

def func2(x):
    return x-1
```

program.py

```
import example

print(example.y)

print(example.func1(2))

print(example.func2(2))
```

Import with Renaming

- You can create an alias when you import a module, by using the **as** keyword.

example.py

```
y=8

def func1(x):
    return 2*x

def func2(x):
    return x-1
```

program.py

```
import example as ex

print(ex.y)

print(ex.func1(2))

print(ex.func2(2))
```

From-Import statement

- We can **import specific names** from a module without importing the module as a whole.

example.py

```
y=8

def func1(x):
    return 2*x

def func2(x):
    return x-1
```

program.py

```
from example import func1

func1(2)
```

Import all Names

- We can **import specific names** from a module without importing the module as a whole.

example.py

```
y=8

def func1(x):
    return 2*x

def func2(x):
    return x-1
```

program.py

```
from example import *

print(y)

print(func1(2))

print(func2(2))
```

sys.path

- sys.path is a built-in variable within the sys module.
- It contains a list of directories that the interpreter will search in for the required module.
- When a **module** is **imported**, the interpreter first searches for the specified module among its **built-in modules**. If not found it **looks through the list of directories defined by sys.path**.

```
# importing module
import sys

# appending a path
sys.path.append('C:/Users/Vanshi/Desktop')

# printing all paths
sys.path
```


Math Module

- part of standard library that contains functions that are useful for performing **mathematical calculations**.
- The `math` module defines variables `pi` and `e`.

```
import math

x = math.ceil(1.4)
y = math.floor(1.4)

print(x) # returns 2
print(y) # returns 1
```

```
# import statement example
# to import standard module math

import math
print("The value of pi is", math.pi)
```

```
The value of pi is 3.141592653589793
```

Math Module functions

math Functions	Description
<code>acos(x)</code>	Returns the arc cosine of x , in radians.
<code>asin(x)</code>	Returns the arc sine of x , in radians.
<code>atan(x)</code>	Returns the arc tangent of x , in radians.
<code>ceil(x)</code>	Returns the smallest integer that is greater than or equal to x .
<code>cos(x)</code>	Returns the cosine of x in radians.
<code>degrees(x)</code>	Assuming x is an angle in radians, the function returns the angle converted to degrees.
<code>exp(x)</code>	Returns e^x
<code>floor(x)</code>	Returns the largest integer that is less than or equal to x .
<code>hypot(x, y)</code>	Returns the length of a hypotenuse that extends from (0, 0) to (x , y).
<code>log(x)</code>	Returns the natural logarithm of x .
<code>log10(x)</code>	Returns the base-10 logarithm of x .
<code>radians(x)</code>	Assuming x is an angle in degrees, the function returns the angle converted to radians.
<code>sin(x)</code>	Returns the sine of x in radians.
<code>sqrt(x)</code>	Returns the square root of x .
<code>tan(x)</code>	Returns the tangent of x in radians.

Random Module

- A built-in module that enable to make random numbers.

Method	Description
<u>seed()</u>	Initialize the random number generator
<u>getstate()</u>	Returns the current internal state of the random number generator
<u>setstate()</u>	Restores the internal state of the random number generator
<u>getrandbits()</u>	Returns a number representing the random bits
<u>randrange()</u>	Returns a random number between the given range
<u>randint()</u>	Returns a random number between the given range
<u>choice()</u>	Returns a random element from the given sequence
<u>choices()</u>	Returns a list with a random selection from the given sequence
<u>shuffle()</u>	Takes a sequence and returns the sequence in a random order
<u>sample()</u>	Returns a given sample of a sequence
<u>random()</u>	Returns a random float number between 0 and 1
<u>uniform()</u>	Returns a random float number between two given parameters

<https://docs.python.org/3/library/random.html>

Random Module

```
import random

# random number from 0 to 1
print(random.random())
# Output 0.16123124494385477

# random number from 10 to 20
print(random.randint(10, 20))
# Output 18

# random number from 10 to 20 with step 2
print(random.randrange(10, 20, 2))
# Output 14

# random float number within a range
print(random.uniform(5.5, 25.5))
# Output 5.86390810771935

# random choice from sequence
print(random.choice([10, 20, 30, 40, 50]))
# Output 30

# random sample from sequence
print(random.sample([10, 20, 30, 40, 50], k=3))
# Output [50, 10, 20]
```

```
# random sample with replacement
print(random.choices([10, 20, 30, 40, 50], k=3))
# Output [30, 10, 40]

# random shuffle
x = [10, 20, 30, 40, 50, 60]
random.shuffle(x)
print(x)
# [60, 10, 30, 20, 50, 40]

# random seed
random.seed(2)
print(random.randint(10, 20))
# 10

random.seed(2)
print(random.randint(10, 20))
# 10
```

- Seed value: initializes the formula that generates random numbers

Random Module

- Seed value: initializes the formula that generates random numbers

$g(\text{seed1})=x1$ $g(\text{seed2})=y1$ $g(\text{seed3})=z1$

$g(x1)=x2$ $g(y1)=y2$ $g(z1)=z2$

$g(x2)=x3$ $g(y2)=y3$ $g(z2)=z3$

Random Module

- The **random** method returns a random float number between 0.0 to 1.0.
- The **randint** method returns a random integer between the specified integers.
- The **randrange** method returns a randomly selected element from the range created by the **start**, **stop** and **step** arguments.
- The value of start is 0 and the value of step is 1 by default.

Example: random()

```
>>> import random
>>> random.random()
0.645173684807533
```

Example: randint()

```
>>> import random
>>> random.randint(1, 100)
95
>>> random.randint(1, 100)
49
```

Example:

```
>>> random.randrange(1, 10)
2
>>> random.randrange(1, 10, 2)
5
>>> random.randrange(0, 101, 10)
80
```

Random Module

- The **choice** method returns a randomly selected element from a non-empty sequence.
- An empty sequence as argument raises an IndexError.

Example:

```
>>> import random
>>> random.choice('computer')
't'
>>> random.choice([12,23,45,67,65,43])
45
>>> random.choice((12,23,45,67,65,43))
67
```

- The **shuffle** method randomly reorders the elements

Example:

```
>>> numbers=[12,23,45,67,65,43]
>>> random.shuffle(numbers)
>>> numbers
[23, 12, 43, 65, 67, 45]
>>> random.shuffle(numbers)
>>> numbers
[23, 43, 65, 45, 12, 67]
```

Conditional Execution

- It is possible to create a **module** that can be **run** as a **standalone program** or **imported** into another program

example.py

```
def main():  
    print("main example")  
    print(func1(5))  
  
def func1(x):  
    return 2*x  
  
def func2(x):  
    return x-1  
  
main()
```

program.py

```
import example  
  
def main():  
    print("main program")  
    print(example.func1(2))  
  
main()
```

```
main example  
10  
main program  
4
```


Conditional Execution

- You can write each **module** so it executes **its main** function only when the module is **being run as the main program**.
- When a **source code** file is loaded into the Python **interpreter**, a special variable called **__name__** is created.
- If the source code file has been **imported as a module**, the **__name__** variable will be set to the **name of the module**.
- If the source code file is being **executed as the main** program, the **__name__** variable will be set to the value **'__main__'**.

Conditional Execution

example.py

```
def main():  
    print("main example")  
    print(func1(5))  
  
def func1(x):  
    return 2*x  
  
def func2(x):  
    return x-1  
  
if __name__ == "__main__":  
    main()
```

program.py

```
import example  
  
def main():  
    print("main program")  
    print(example.func1(2))  
  
main()
```

```
main program  
4
```

Exception Handling

- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.

```
try:  
    print(x)  
except:  
    print("An exception occurred")
```

```
try:  
    print(x)  
except NameError:  
    print("Variable x is not defined")  
except:  
    print("Something else went wrong")
```

Exception Handling

- The **else** block lets you execute code when there is no error.
- The **finally** block lets you execute code, regardless of the result of the try- and except blocks.

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

Exception Types

- `ArithmeticError`
 - `FloatingPointError`
 - `OverflowError`
 - `ZeroDivisionError`
- `EOFError`
- `ImportError`
 - `ModuleNotFoundError`
- `LookupError`
 - `IndexError`
 - `KeyError`
- `FileNotFoundError`
- `ValueError`
- `Exception`

<https://airbrake.io/blog/python-exception-handling/class-hierarchy>

Exception Example

```
def get_int_from_user():  
    try:  
        num= int(input('Enter a number: \n'))  
    except ValueError as err:  
        print (err)  
        num=0  
    else:  
        num=num+5  
    finally:  
        return num
```

print Exception and raise

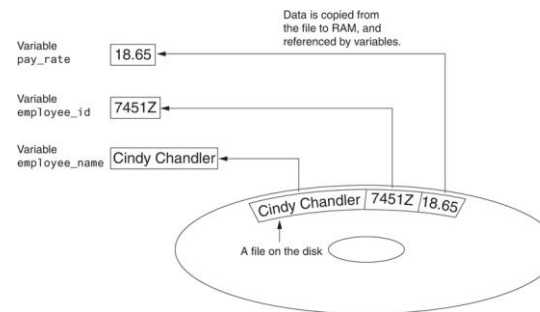
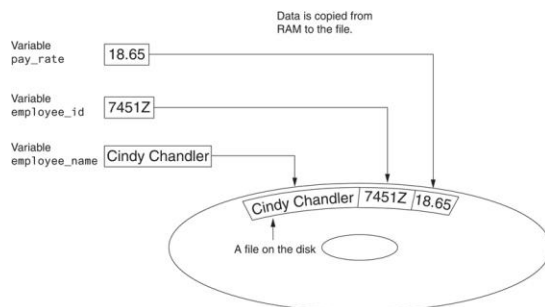
- Example:

```
>>> try:
...     a = int(input("Enter a positive integer: "))
...     if a <= 0:
...         raise ValueError("That is not a positive number!")
... except ValueError as ve:
...     print(ve)
...
```

```
Enter a positive integer: -2
That is not a positive number!
```

Intro to Files

- For program to retain data between runs, you must save the data.
- Data is saved to a file, typically on computer disk.
- Saved data can be retrieved and used at a later time
 - “Writing data to”: saving data on a file.
 - Output file: a file that data is written to.
 - “Reading data from”: reading data from a file.



Types of Files

- In general, two types of files.
 - Text file: contains data that has been encoded as text.
 - Binary file: contains data that has not been converted to text.
- Filename extensions: short sequences of characters that appear at the end of a filename preceded by a period.
 - Extension indicates type of data stored in the file.
- Two ways to access data stored in file.
 - Sequential access: file read sequentially from beginning to end, can't skip ahead.
 - Direct access: can jump directly to any piece of data in the file.

Working with Files

- File object: object associated with a specific file.
 - Provides a way for a program to work with the file: file object referenced by a variable.
 - Open `file_object = open(filename, mode)`
 - Work `file_object.write(string)`
`file_object.read()`
`for line in file_object:`
`statements`
 - Close `file_object.close()`

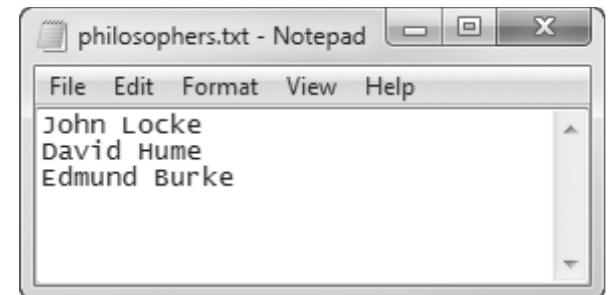
File Modes

Mode	Description
'r'	This is the default mode. It Opens file for reading.
'w'	This Mode Opens file for writing. If file does not exist, it creates a new file. If file exists it truncates the file.
'x'	Creates a new file. If file already exists, the operation fails.
'a'	Open file in append mode. If file does not exist, it creates a new file.
't'	This is the default mode. It opens in text mode.
'b'	This opens in binary mode.
'+'	This will open a file for reading and writing (updating)

Writing to File Example

Program 6-1 (file_write.py)

```
1 # This program writes three lines of data
2 # to a file.
3 def main():
4     # Open a file named philosophers.txt.
5     outfile = open('philosophers.txt', 'w')
6
7     # Write the names of three philosophers
8     # to the file.
9     outfile.write('John Locke\n')
10    outfile.write('David Hume\n')
11    outfile.write('Edmund Burke\n')
12
13    # Close the file.
14    outfile.close()
15
16 # Call the main function.
17 main()
```



Files Examples

```
file_handle=open("c:\\temp\\test.txt","w")
file_handle.write("This is a lecture about Python.\n")
file_handle.write("Python is a very nice language.\n")
file_handle.write("In Python everything is object\n")
file_handle.close()
```

WARNING!

Remember, when you use the 'w' mode, you are creating the file on the disk. If a file with the specified name already exists when the file is opened, the contents of the existing file will be deleted

Reading from File Example

- read method: file object method that reads entire file contents into memory

Program 6-2 (file_read.py)

```
1 # This program reads and displays the contents
2 # of the philosophers.txt file.
3 def main():
4     # Open a file named philosophers.txt.
5     infile = open('philosophers.txt', 'r')
6
7     # Read the file's contents.
8     file_contents = infile.read()
9
10    # Close the file.
11    infile.close()
12
13    # Print the data that was read into
14    # memory.
15    print(file_contents)
16
17 # Call the main function.
18 main()
```

Program Output

John Locke

David Hume

Edmund Burke

Reading from File Example

- readline method: file object method that reads a line from the file

Program 6-3 (line_read.py)

```
1 # This program reads the contents of the
2 # philosophers.txt file one line at a time.
3 def main():
4     # Open a file named philosophers.txt.
5     infile = open('philosophers.txt', 'r')
6
7     # Read three lines from the file.
8     line1 = infile.readline()
9     line2 = infile.readline()
10    line3 = infile.readline()
11
12    # Close the file.
13    infile.close()
14
15    # Print the data that was read into
16    # memory.
17    print(line1)
18    print(line2)
19    print(line3)
20
21 # Call the main function.
22 main()
```



Program Output

John Locke

David Hume

Edmund Burke

Stripping Example

Program 6-5 (strip_newline.py)

```
1 # This program reads the contents of the
2 # philosophers.txt file one line at a time.
3 def main():
4     # Open a file named philosophers.txt.
5     infile = open('philosophers.txt', 'r')
6
7     # Read three lines from the file.
8     line1 = infile.readline()
9     line2 = infile.readline()
10    line3 = infile.readline()
11
12    # Strip the \n from each string.
13    line1 = line1.rstrip('\n')
14    line2 = line2.rstrip('\n')
15    line3 = line3.rstrip('\n')
16
17    # Close the file.
18    infile.close()
19
20    # Print the data that was read into
21    # memory.
22    print(line1)
23    print(line2)
24    print(line3)
25
26 # Call the main function.
27 main()
```

rstrip - Remove white spaces at the end of the string.

Program Output

John Locke
David Hume
Edmund Burke

Appending Example

Let's assume the file **friends.txt** contains the following names, each in a separate line:

Joe
Rose
Geri

The following code opens the file and appends additional data to its existing contents.

```
myfile = open('friends.txt', 'a')  
myfile.write('Matt\n')  
myfile.write('Chris\n')  
myfile.write('Suze\n')  
myfile.close()
```

After this program runs, the file friends.txt will contain the following data:

Joe
Rose
Geri
Matt
Chris
Suze

Writing Numeric Data to File

Program 6-6 (write_numbers.py)

```
1 # This program demonstrates how numbers
2 # must be converted to strings before they
3 # are written to a text file.
4
5 def main():
6     # Open a file for writing.
7     outfile = open('numbers.txt', 'w')
8
9     # Get three numbers from the user.
10    num1 = int(input('Enter a number: '))
11    num2 = int(input('Enter another number: '))
12    num3 = int(input('Enter another number: '))
13
14    # Write the numbers to the file.
15    outfile.write(str(num1) + '\n')
16    outfile.write(str(num2) + '\n')
17    outfile.write(str(num3) + '\n')
18
19    # Close the file.
20    outfile.close()
21    print('Data written to numbers.txt')
22
23 # Call the main function.
24 main()
```

- Numbers must be converted to strings before they are written to a file.



Program Output

Enter a number: **22** ↵

Enter another number: **14** ↵

Enter another number: **-99** ↵

Data written to numbers.txt

Reading Numeric Data from a File

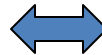
```
1 infile = open('numbers.txt', 'r')  
2 value = infile.readline()  
3 infile.close()
```



When you read numbers from a text file, they are always read as strings



```
1 infile = open('numbers.txt', 'r')  
2 string_input = infile.readline()  
3 value = int(string_input)  
4 infile.close()
```



```
1 infile = open('numbers.txt', 'r')  
2 value = int(infile.readline())  
3 infile.close()
```

R/W Numeric Data to File

- Write numbers to file.

```
file_handle=open('test.txt','w')
for i in range(5):
    num1=int(input('Enter a number:\n'))
    file_handle.write(str(num1)+'\n')
file_handle.close()
```

- Read numbers from file.

```
file_handle=open('test.txt','r')
for line in file_handle:
    num=int(line)
    print(num)
file_handle.close()
```

Read unknown number of Lines

Program 6-10 (read_sales2.py)

```
1 # This program uses the for loop to read
2 # all of the values in the sales.txt file.
3
4 def main():
5     # Open the sales.txt file for reading.
6     sales_file = open('sales.txt', 'r')
7
8     # Read all the lines from the file.
9     for line in sales_file:
10         # Convert line to a float.
11         amount = float(line)
12         # Format and display the amount.
13         print(format(amount, '.2f'))
14
15     # Close the file.
16     sales_file.close()
17
18 # Call the main function.
19 main()
```

Program Output

1000.00

2000.00

3000.00

4000.00

5000.00



Read unknown number of Lines

```
1 # This program reads all of the values in
2 # the sales.txt file.
3
4 def main():
5     # Open the sales.txt file for reading.
6     sales_file = open('sales.txt', 'r')
7
8     # Read the first line from the file, but
9     # don't convert to a number yet. We still
10    # need to test for an empty string.
11    line = sales_file.readline()
12
13    # As long as an empty string is not returned
14    # from readline, continue processing.
15    while line != "":
16        # Convert line to a float.
17        amount = float(line)
18
19        # Format and display the amount.
20        print(format(amount, '.2f'))
21
22        # Read the next line.
23        line = sales_file.readline()
24
25    # Close the file.
26    sales_file.close()
27
28 # Call the main function.
29 main()
```

Program Output

1000.00

2000.00

3000.00

4000.00

5000.00



Record Processing

- Record: set of data that describes one item.
- Field: single piece of data within a record.
- Write record to sequential access file by writing the fields one after the other.
- Read record from sequential access file by reading each field until record complete.

Writing Records

```
1 # This program gets employee data from the user and
2 # saves it as records in the employee.txt file.
3
4 def main():
5     # Get the number of employee records to create.
6     num_emps = int(input('How many employee records ' +
7                           'do you want to create? '))
8
9     # Open a file for writing.
10    emp_file = open('employees.txt', 'w')
11
12    # Get each employee's data and write it to
13    # the file.
14    for count in range(1, num_emps + 1):
15        # Get the data for an employee.
16        print('Enter data for employee #', count, sep='')
17        name = input('Name: ')
18        id_num = input('ID number: ')
19        dept = input('Department: ')
20
21        # Write the data as a record to the file.
22        emp_file.write(name + '\n')
23        emp_file.write(id_num + '\n')
24        emp_file.write(dept + '\n')
25
26        # Display a blank line.
27        print()
28
29    # Close the file.
30    emp_file.close()
31    print('Employee records written to employees.txt.')
32
33 # Call the main function.
34 main()
```

Program Output (with input shown in bold)

How many employee records do you want to create?
Enter the data for employee #1

Name: **Ingrid Virgo** ↵

ID number: **4587** ↵

Department: **Engineering** ↵

Enter the data for employee #2

Name: **Julia Rich** ↵

➡ ID number: **4588** ↵

Department: **Research** ↵

Enter the data for employee #3

Name: **Greg Young** ↵

ID number: **4589** ↵

Department: **Marketing** ↵

Employee records written to employees.txt


```

1 # This program displays the records that are
2 # in the employees.txt file.
3
4 def main():
5     # Open the employees.txt file.
6     emp_file = open('employees.txt', 'r')
7
8     # Read the first line from the file, which is
9     # the name field of the first record.
10    name = emp_file.readline()
11
12    # If a field was read, continue processing.
13    while name != "":
14        # Read the ID number field.
15        id_num = emp_file.readline()
16
17        # Read the department field.
18        dept = emp_file.readline()
19
20        # Strip the newlines from the fields.
21        name = name.rstrip('\n')
22        id_num = id_num.rstrip('\n')
23        dept = dept.rstrip('\n')
24
25        # Display the record.
26        print('Name:', name)
27        print('ID:', id_num)
28        print('Dept:', dept)
29        print()
30
31        # Read the name field of the next record.
32        name = emp_file.readline()
33
34    # Close the file.
35    emp_file.close()
36
37 # Call the main function.
38 main()

```

Reading Records

Program Output

Name: Ingrid Virgo
ID: 4587
Dept: Engineering

Name: Julia Rich
ID: 4588
Dept: Research

Name: Greg Young
ID: 4589
Dept: Marketing



Reading Records

This program displays the records that are
in the employees.txt file.

```
def main():  
    file = open("employees.txt", "r")  
  
    for line in file:  
        list = line.split(",")  
        print(list[0], list[1], list[2], sep = "\t\t")  
  
    file.close()  
  
# Call the main function.  
main()
```

employees.txt

Name, ID, Dept Ingrid Virgo, 4587, Engineering Julia Rich, 4588, Research Greg Young, 4589, Marketing
--

Writing Records

```
def main():
    file = open("employees.txt", "w")

    file.write("Name, ID, Dept\n")    # Write header line
    num_employees = int(input("How many employees do you want to enter? "))

    for i in range(num_employees):
        print("\nEmployee", i + 1)

        name = input("Enter name: ")
        emp_id = input("Enter ID: ")
        dept = input("Enter department: ")

        # Build the line in the format: Name, ID, Dept
        line = ", ".join([name, emp_id, dept])

        file.write(line + "\n")

    file.close()
```

employees.txt

```
Name, ID, Dept
Ingrid Virgo, 4587, Engineering
Julia Rich, 4588, Research
Greg Young, 4589, Marketing
```